ROLE 4 HANDOFF DELIVERABLE • IMMEDIATE OPERATIONAL DIRECTIVE

ROLE 4 → ROLE 1 IMPLEMENTATION HANDOFF SPECIFICATION

Definitive Telemetry Features, Dataset Schema, Behavioral Distributions, Synthetic Simulation Algorithm, Data Leakage Wall, and Pre-Flight Verification Checklist

Source: Role 4 (Research Lead)
Target: Role 1 (Blockchain Simulation Lead)
Status: FINAL & AUTHORITATIVE

Sim Scale: 100 Nodes, 1,000 Tx
Records: 100,000 Event Rows
Output File: validator_behavior_log.csv

Date: September 2026
Classification Wall: true_label strictly hidden
Next Milestone: Role 1 Dataset Export

> **Operational Directive for Role 1**
>
> This document contains the complete, self-contained engineering specification required for Role 1 to implement the blockchain validator behavioral simulation. You do **not** need to read the 25-page Master Research Report to begin coding. All feature names, data types, statistical distributions, pseudocode algorithms, and validation checks in this document are final and aligned with Role 2 (Machine Learning).

1. Definitive Behavioral Feature Set

Following academic literature review (IEEE ICBC 2026, ACM IECC 2019, Wiley 2020), the simulation records exactly **five (5) operational telemetry features** per validator during transaction verification:

Feature Name	Data Type	Allowed Range	Operational Telemetry Definition	Why Role 1 Simulates This Feature
`uptime`	`float64`	$[0.0, 1.0]$	Ratio of operational heartbeats received during the current epoch ($T_{\text{active}} / T_{\text{total}}$).	Quantifies liveness. Distinguishes high-availability servers from intermittent or offline nodes.
`response_time_ms`	`float64`	$[10.0, 1500.0]$	Elapsed time in milliseconds from transaction proposal broadcast to validator signature receipt.	Quantifies execution latency. Detects overloaded nodes, network chokepoints, or deliberate stalling.
`verification_correct`	`int32`	$\{0, 1\}$	Binary agreement ($1 = \text{Correct}$, $0 = \text{Incorrect}$) between validator's local execution and true transaction validity.	Direct signal of Byzantine execution fraud (forging state roots or validating invalid balances).
`transaction_outcome`	`int32`	$\{0, 1\}$	Validator voting decision: $1 = \text{Approved}$, $0 = \text{Declined}$.	Isolates censorship attacks. Malicious nodes deliberately decline valid user transactions.
`suspicious_flag`	`int32`	$\{0, 1\}$	Rule-based peer anomaly trigger: $1 = \text{Flagged}$, $0 = \text{Normal}$.	Simulates peer sanity checks: detecting malformed headers, invalid ECDSA nonces, or double-voting.

2. Exact Dataset Schema: `validator_behavior_log.csv`

Role 1 must export simulation results into a standard comma-separated file named `validator_behavior_log.csv`. The dataset must conform exactly to the 9-column schema below:

Column Name	Pandas Type	Null Policy	Visibility to ML Model	Operational Role & Validation Rule
`transaction_id`	`int64`	NOT NULL	EXCLUDED	Monotonically increasing transaction sequence index ($1, 2, \dots, 1000$).
`validator_id`	`int64`	NOT NULL	EXCLUDED	Unique validator node identifier ($1, 2, \dots, 100$).
`stake`	`float64`	NOT NULL	POST-ML ONLY	Validator locked capital ($10.0 \le S_i \le 1000.0$). Used in $W = S \times T$, NOT as ML input.
`uptime`	`float64`	NOT NULL	ML FEATURE X[0]	Floating point clamped strictly to interval $[0.000, 1.000]$.
`response_time_ms`	`float64`	NOT NULL	ML FEATURE X[1]	Continuous positive latency value clamped to $[10.0, 1500.0]$.
`verification_correct`	`int32`	NOT NULL	ML FEATURE X[2]	Binary discrete integer: exactly 0 or 1.
`transaction_outcome`	`int32`	NOT NULL	ML FEATURE X[3]	Binary discrete integer: exactly 0 or 1.
`suspicious_flag`	`int32`	NOT NULL	ML FEATURE X[4]	Binary discrete integer: exactly 0 or 1.
`true_label`	`int32`	NOT NULL	STRICTLY HIDDEN	Ground-truth status ($1 = \text{Honest}, 0 = \text{Malicious}$). Evaluation ONLY!

The Data Leakage Prevention Protocol

CRITICAL HANDOFF MANDATE: The column `true_label` is the ground-truth assignment created at simulation initialization. It must **NEVER** be passed into the feature matrix `X` given to Role 2's classifier.

Role 1 must either:

1. Export two separate files: `validator_features.csv` (containing columns 1–8) and `validator_ground_truth.csv` (containing `validator_id` and `true_label`), OR
2. Export `validator_behavior_log.csv` with explicit instructions for Role 2 to execute: `X = df[['uptime', 'response_time_ms', 'verification_correct', 'transaction_outcome', 'suspicious_flag']]` and `y_eval = df['true_label']`.

3. Behavioral Simulation Guidance: Honest vs. Malicious Nodes

3.1 Validator Pool Composition

- **Total Population (N):** 100 Validators ($v_1 \dots v_{100}$).
- **Honest Nodes (N_H):** 80 Validators (80%) with `true_label = 1`.
- **Malicious Nodes (N_M):** 20 Validators (20%) with `true_label = 0`.
- **Stake Distribution:** Initialized via a Pareto/Power-Law or bounded log-normal distribution ($S_i \in [10.0, 1000.0]$), reflecting real-world capital inequality where a minority of whales hold large stake. Both honest and malicious cohorts should possess high-, medium-, and low-stake nodes.

3.2 Concrete Statistical Generation Parameters

To ensure scientific realism, the distributions **must overlap**. Honest nodes experience occasional network faults; malicious nodes strategically camouflage their behavior by validating honestly most of the time.

Feature	Honest Generation Rule ($v_i \in \text{Honest}$)	Malicious Generation Rule ($v_i \in \text{Malicious}$)	Basis: Literature vs. Assumption
`uptime`	$X \sim \mathcal{N}(\mu=0.98, \sigma=0.015)$ Clamp to $[0.85, 1.00]$	$X \sim \text{Beta}(\alpha=4.0, \beta=1.5)$ Scale & clamp to $[0.40, 0.95]$ (Mean ≈ 0.73)	**Literature Supported:** High uptime baseline with log-tail drops (Harris, IEEE ICBC 2026).
`response_time_ms`	$X \sim \text{LogNormal}(\mu=3.8, \sigma=0.35)$ Clamp to $[15.0, 250.0]$ (Median $\approx 45\text{ ms}$)	$X \sim \text{LogNormal}(\mu=5.3, \sigma=0.60)$ Clamp to $[50.0, 1200.0]$ (Median $\approx 200\text{ ms}$)	**Literature Supported:** Log-normal queuing delay in distributed P2P mempools (Windiatmaja, 2023).
`verification_correct`	$P(\text{Correct} = 1) = 0.985$ (1.5% accidental parse/timeout errors)	$P(\text{Correct} = 1) = 0.680$ (32.0% intentional Byzantine fraud)	**Design Assumption:** Calibrated to model Byzantine camouflage (on-off strategic attacks).
`transaction_outcome`	$P(\text{Approve} = 1) = 0.940$ (6% legitimate rejections of bad txs)	$P(\text{Approve} = 1) = 0.720$ (28% selective censorship rejections)	**Design Assumption:** Reflects censorship activity without causing immediate network eviction.
`suspicious_flag`	$P(\text{Flag} = 1) = 0.020$ (2.0% network false-alarm rate)	$P(\text{Flag} = 1) = 0.320$ (32.0% true anomaly detection rate)	**Design Assumption:** Realistic anomaly detector precision based on intrusion telemetry.

4. Step-by-Step Synthetic Simulation Algorithm

Role 1 can implement the data generator using the following structured Python algorithm:

```
import numpy as np
import pandas as pd

def generate_blockchain_telemetry(n_validators=100, n_transactions=1000, seed=42):
    np.random.seed(seed)

    # 1. Initialize Validator Population
    n_malicious = int(n_validators * 0.20)  # 20 malicious, 80 honest
    labels = np.array([0] * n_malicious + [1] * (n_validators - n_malicious))
    np.random.shuffle(labels)

    # Pareto/log-normal stake distribution ($10 to $1000)
    stakes = np.clip(np.random.pareto(a=1.5, size=n_validators) * 150 + 20, 10.0, 1000.0)

    records = []

    # 2. Simulate 1,000 Transaction Events
    for tx_id in range(1, n_transactions + 1):
        for v_id in range(1, n_validators + 1):
            is_honest = labels[v_id - 1]
            stake = stakes[v_id - 1]

            if is_honest == 1:
                uptime = np.clip(np.random.normal(0.98, 0.015), 0.85, 1.00)
                resp_time = np.clip(np.random.lognormal(3.8, 0.35), 15.0, 300.0)
                verif_correct = np.random.choice([1, 0], p=[0.985, 0.015])
                tx_outcome = np.random.choice([1, 0], p=[0.940, 0.060])
                susp_flag = np.random.choice([1, 0], p=[0.020, 0.980])
            else:
                uptime = np.clip(np.random.beta(4.0, 1.5), 0.40, 0.95)
                resp_time = np.clip(np.random.lognormal(5.3, 0.60), 40.0, 1400.0)
                verif_correct = np.random.choice([1, 0], p=[0.680, 0.320])
                tx_outcome = np.random.choice([1, 0], p=[0.720, 0.280])
                susp_flag = np.random.choice([1, 0], p=[0.320, 0.680])

            records.append({
                'transaction_id': tx_id,
                'validator_id': v_id,
                'stake': round(float(stake), 2),
                'uptime': round(float(uptime), 4),
                'response_time_ms': round(float(resp_time), 2),
                'verification_correct': int(verif_correct),
                'transaction_outcome': int(tx_outcome),
                'suspicious_flag': int(susp_flag),
                'true_label': int(is_honest)
            })

    df = pd.DataFrame(records)
    df.to_csv('validator_behavior_log.csv', index=False)
    print(f"Generated {len(df)} records across {n_validators} validators and {n_transactions} transactions."
    return df
```

5. Data Quality Requirements & Integrity Standards

- **Strictly Zero Missing Values:** The exported CSV must contain zero `NaN`, `null`, or empty string values. If a simulated node is offline, assign `uptime = 0.0` and `response_time_ms = 1500.0` (timeout cap).

- **Strict Clamping & No Impossible Values:** Probabilities and ratios must remain in $[0.0, 1.0]$. Latencies must remain positive (>0). Discrete flags must contain only $\{0, 1\}$.
- **No Perfect Separability:** Do not adjust malicious parameters to 0.0 or honest parameters to 1.0. If a simple rule like `if response_time > 100: malicious` achieves 100% accuracy, the dataset fails scientific validation.
- **Deterministic Reproducibility:** Role 1 must fix and log the random seed (e.g., `seed = 42`) so that all comparative evaluation runs across Roles 1, 2, and 3 use identical behavioral trajectories.

6. Pre-Flight Implementation Checklist for Role 1

Role 1 Handoff Verification Checklist

Before declaring the simulation phase complete and handing off data to Role 2, Role 1 must verify:

✓ Exactly **100 validator entities** created (80 honest, 20 malicious).

✓ Heterogeneous capital stakes assigned ($10.0 \le S_i \le 1000.0$) across both cohorts.

✓ Hidden ground truth (`true_label`) assigned and documented as strictly evaluation-only.

✓ Exactly **1,000 transactions** simulated sequentially.

✓ Total record count equals exactly $100 \times 1000 = 100,000$ rows.

✓ All five required features recorded: `uptime` , `response_time_ms` , `verification_correct` , `transaction_outcome` , `suspicious_flag` .

✓ Verified zero missing, null, or out-of-bound values in the exported dataset.

✓ Distributions verified to have realistic statistical overlap (no trivial linear separation).

✓ Dataset exported as `validator_behavior_log.csv` .

✓ Random seed logged for deterministic reproduction by Role 2 and Role 3.